SECTION 4: ARRAYS

Lecture 14: Declaring Arrays

One dimensional arrays

type, DIMENSION(lower:upper) :: array1_name, …

type :: array1_name(lower:upper), …

For Example:

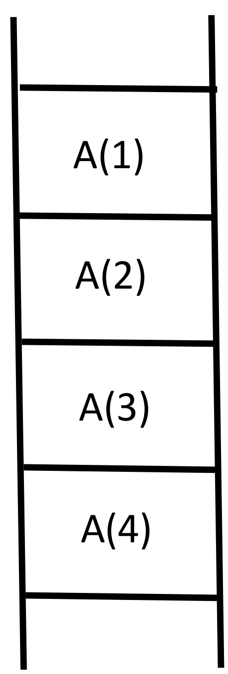
INTEGER, DIMENSION(1:100) :: day_of_year

REAL, DIMENSION(-30:30) :: array1, array2

REAL :: array1(-30:30), array2(0:100)

LOGICAL, DIMENSION(20) :: new_array

Array elements occupy successive locations in computer memory.



Two-Dimensional Arrays (or Rank 2 Array)

type, DIMENSION(lower1:upper1, lower2:upper2) :: array1_name, …

type :: array1_name(lower1:upper1, lower2:upper2), …

For instance:

INTEGER, PARAMETER :: NX=50, NY=50

REAL, DIMENSION(0:NX+1, 0:NY+1) :: phi

REAL :: temperature(1:10, 1:20)

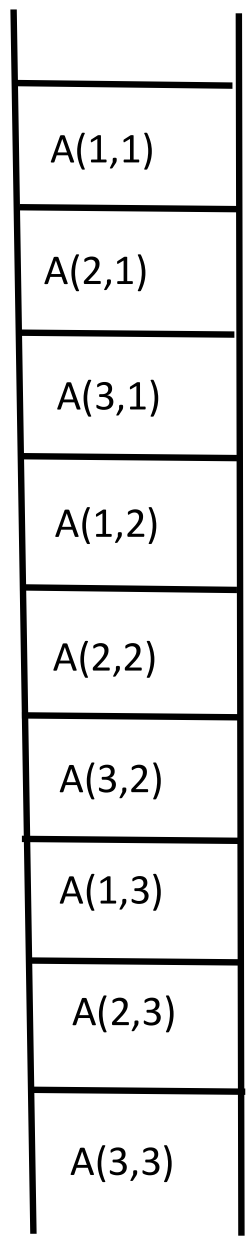
Multidimensional arrays stored in **column-major order** (C language is row-major)

That is for a 3x3 element array A

A(1,1)	A(1,2)	A(1,3)
A(2,1)	A(2,2)	A(2,3)
A(3,1)	A(3,2)	A(3,3)

Stored in memory sequence as:

A(1,1) A(2,1) A(3,1) A(1,2) A(2,2) A(3,2) A(1,3) A(2,3) A(3,3)



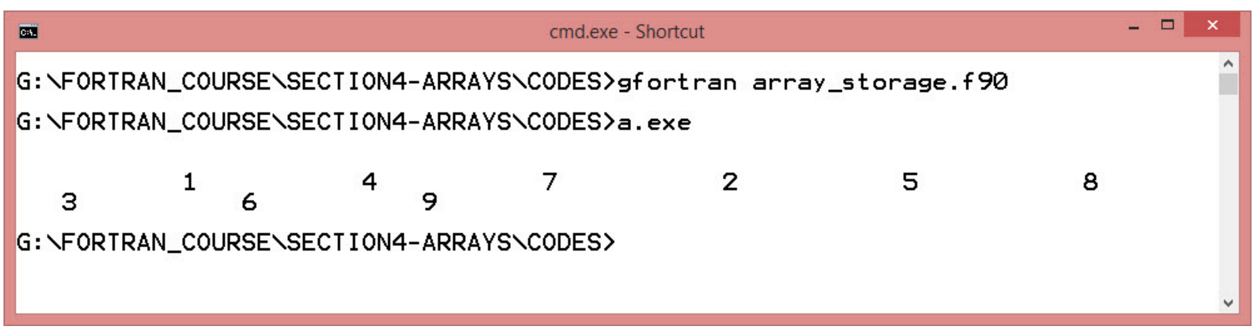
Example Next Page:

```fortran
PROGRAM array_storage
IMPLICIT none
INTEGER :: a(3,3)

a(1,1)=1; a(1,2)=2; a(1,3)=3
a(2,1)=4; a(2,2)=5; a(2,3)=6
a(3,1)=7; a(3,2)=8; a(3,3)=9

PRINT*; PRINT*
PRINT*,a

END PROGRAM array_storage
```



Lecture 15: Using Arrays

Back to our very first example code from SECTION 1

Compute the temperature distribution in a plate using a finite-difference approach.

```
!       COMPUTE TEMPERATURE DISTRIBUTION IN PLATE

        PROGRAM EXAMPLE
        IMPLICIT NONE
        INTEGER, PARAMETER :: NX=3,NY=3
        INTEGER :: i,j, outeriterations
        REAL, DIMENSION(0:NX+1,0:NY+1) :: t

!       INITIALIZE TEMPERATURE ARRAY, t
        t=0.0
!       ADD BOUNDARY CONDITIONS
        t(:,0)=100.;  t(0,:)=100.

        DO OUTERITERATIONS=1,100
!           DO i=1,NX
!              DO j=1,NY
!                t(i,j)=0.25*(t(i+1,j)+t(i-1,j)+t(i,j+1)+t(i,j-1))
                 t(1:NX,1:NY)=0.25*(t(2:NX+1,1:NY)+t(0:NX-1,1:NY)+ &
                                    t(1:NX,2:NY+1)+t(1:NX,0:NY-1))
!              END DO
!           END DO
        END DO
        PRINT*; PRINT*
        DO j=NY+1,0,-1
           WRITE(*,100)( t(i,j),i=0,NX+1)
           PRINT*
        END DO
100     FORMAT(20F8.2)

        STOP
        END PROGRAM EXAMPLE
```

cmd.exe - Shortcut

```
G:\FORTRAN_COURSE\SECTION4-ARRAYS\CODES>a.exe

  100.00    0.00    0.00    0.00    0.00
  100.00   50.00   28.57   14.29    0.00
  100.00   71.43   50.00   28.57    0.00
  100.00   85.71   71.43   50.00    0.00
  100.00  100.00  100.00  100.00  100.00

G:\FORTRAN_COURSE\SECTION4-ARRAYS\CODES>
```

Lecture 16: Intrinsic Functions for Matrix Operations

MATMUL(MATRIX_1, MATRIX_2)

DOT_PRODUCT(VECTOR_1, VECTOR_2)

TRANSPOSE(MATRIX)

MAXVAL(ARRAY, MASK)

MINVAL(ARRAY, MASK)

Example Including the Use of a Mask:

```
!     EXAMPLE ARRAY MASK USEAGE
      IMPLICIT none
      INTEGER, PARAMETER :: nx=3, ny=3
      INTEGER :: i,j
      REAL, DIMENSION(NX,NY) :: a=0, b=0, c=0
      LOGICAL, DIMENSION(NX,NY) :: msk=.true.
      a(1,1)=1; a(1,2)=2; a(1,3)=3
      a(2,1)=4; a(2,2)=5; a(2,3)=6
      a(3,1)=7; a(3,2)=8; a(3,3)=9
!     identity matrix
      b(1,1)=1; b(1,2)=0; b(1,3)=0
      b(2,1)=0; b(2,2)=1; b(2,3)=0
      b(3,1)=0; b(3,2)=0; b(3,3)=1
      msk(1,1)=.true.; msk(1,2)=.true.; msk(1,3)=.true.
      msk(2,1)=.true.; msk(2,2)=.true.; msk(2,3)=.true.
      msk(3,1)=.true.; msk(3,2)=.true.; msk(3,3)=.false.
      PRINT*
      PRINT*,'Max Value:',MAXVAL(a, MASK = msk)
      PRINT*,'Max Value:',MAXVAL(a, MASK = a .lt. 5.)
      c=MATMUL(a,b)
      PRINT*
      do i=1,NX
          WRITE(*,'(4F10.1)')(c(i,j),j=1,NY)
      end do
      END
```

cmd.exe - Shortcut

```
G:\FORTRAN_COURSE\SECTION4-ARRAYS\CODES>a.exe

 Max Value: 8.
 Max Value: 4.

       1.0       2.0       3.0
       4.0       5.0       6.0
       7.0       8.0       9.0

G:\FORTRAN_COURSE\SECTION4-ARRAYS\CODES>
```